Matrix multiplication on RePair-compressed matrices, on a GPU (g-mm-repair)

Repository: https://github.com/ftosoni/g-mm-repair

PDF version of this README: README.pdf



A high-performance level-synchronous GPU executor (written in CUDA C++) for computing right matrix-vector multiplication $y = Mx$ over grammar-compressed matrices. It implements a double-buffered level sweep algorithm with an **"emit-on-the-spot"** memory optimization that avoids carrying intermediate rule expansions to the top level, drastically reducing GPU memory usage and overhead.

This repository implements the GPU-acceleration techniques described in the manuscript:

> **"Streaming Right Multiplication over Grammar-Compressed Matrices: A Memory-Bounded GPU Engine for Genotype and Graph Data"** — Francesco Tosoni and Gabriele Mencagli, *SIAM Symposium on Algorithm Engineering and Experiments (ALENEX 2027)*, to appear.

To regenerate the paper's tables and figures, the Reproducibility section below is all you need (a single `./runme.sh`). **REPRODUCIBILITY.md** collects further details: per-experiment manual commands, baseline setup, dataset provenance and file formats.

> **Table and figure numbers.** Throughout this repository (this README, REPRODUCIBILITY.md, `reproduce.sh`, `extract_results.py`, `manuscript/`), table, figure and section numbers refer to the full version of the paper on arXiv, **arXiv:2607.24971v1**, which also contains the supplementary material that the ALENEX 2027 proceedings omit for space (Figure A.1 and Table B.1). The camera-ready numbers the measured results differently:

arXiv v1	Camera-ready	Content
Table 4.1	Table 6.1	Genotype structural figures
Table 4.2	Table 6.2	Genotype time per vector (the camera-ready's last column is the speed-up over cuSPARSE, from the same data)
Table 4.3	Table 6.3	Genotype space and energy vs cuSPARSE
Figure 4.3	Figure 6.1	Device footprint vs non-zeros
Table 5.1	Table 6.6	Largest graphs (10M–1.2G edges)
Table 5.2	Table 6.7	Wikidata Boolean and Tropical product
Table A.1	Table 6.4	Batched right product (SpMM)
Figure A.1	— (arXiv only)	Batched throughput vs batch size B
Table B.1	— (arXiv only)	Host-side construction cost
Table B.2	Table 6.5	Wikidata structural figures

🏅 Artifact Availability and Reproducibility (ALENEX 2027)

The version of the code evaluated by the ALENEX 2027 Artifact Evaluation Committee, and the data it consumes, are permanently archived:

code 10.5281/zenodo.23139248 data 10.5281/zenodo.22677747
archived swh:1:dir:ef2c5f738ef173b38a2c7f1edc8969d06159785d

Code	Zenodo, DOI `10.5281/zenodo.23139248`: `g-mm-repair.tar.gz`, including the `mm-repair` submodule
Data	Zenodo, DOI `10.5281/zenodo.22677747`: datasets and RePair grammars (see Data Availability)
Version	Git tag `alenex27-ae`, commit `f5bce98` (submodule `mm-repair` at `ee2e03b`)
Archive MD5	`2f2325848f1f48eed2a116334a3b2d25` (same file on Zenodo and on the GitHub release)
Source snapshot	Software Heritage, revision `f007357`: same code as `f5bce98`, which only updates the SWHID quoted in this README

To reproduce the paper from the archived code: `tar xzf g-mm-repair.tar.gz && cd g-mm-repair && ./runme.sh` (see Reproducibility).

✨ Key Features

- ⚡ **Double-Buffered Level Sweep**: Computes frontiers in parallel, level-by-level, utilizing only two active frontier buffers on the GPU.
- 💡 **Emit-on-the-Spot Optimization**: Accumulates non-terminal rules directly into the output vector y as soon as they are computed, eliminating intermediate dummy pass-through nodes and saving memory.
- 🧠 **Advanced Semiring Support**: Generalizes to Boolean (graph reachability) and Tropical (single-source shortest paths) semirings via compile-time/runtime configuration.
- 📦 **OOM-Resistant Host Scheduler**: Employs a flat 1D host memory mapping to drastically reduce memory allocation overhead for large grammar trees before launching CUDA execution.
- 🛠️ **Seamless Integration**: Works directly with RePair compression outputs from `mm-repair`.

📁 Project Structure

```
g-mm-repair/
├── gpu-engine/             # Core GPU engine implementation
│   ├── gpu_engine.cu       # CUDA kernels (double-buffered sweep & emit-on-the-spot)
│   ├── gpu_engine.h        # Struct definitions and engine entry points
│   ├── gpu_engine_test.cu  # Verification and benchmark runner
│   ├── cusparse_test.cu    # Baseline cuSPARSE implementation
│   └── Makefile            # GPU engine build configuration
├── mm-repair/              # Submodule: CPU-based grammar compression
├── manuscript/             # LaTeX sources, tables, figures, and execution logs
│   ├── logs/               # Target directory for execution logs (gitignored)
│   ├── tables/             # LaTeX tables (generated by extract_results.py)
│   └── figures/            # TikZ figures and data
├── reproduce.sh            # Complete end-to-end experiment replication orchestrator
├── extract_results.py      # Automated table/figure data extraction from logs
├── prepare_bio_datasets.py # Automated genotype data fetcher & simulator
├── generate_msprime.py     # Coalescent population simulator for synthetic haplotypes
├── process_wikidata.py     # Wikidata relation sparse extractor
├── README.md               # Quickstart and overview
└── REPRODUCIBILITY.md      # Detailed replication guide
```

🚀 Quick Start

Prerequisites. A single NVIDIA GPU with the CUDA Toolkit ≥ 12.0 (`nvcc` , cuSPARSE, NVML); a C++17 compiler with OpenMP (`g++`) and GNU `make` ; SDSL-lite; Python 3 with `venv` ; and the command-line tools `git` , `curl` , `tar` , `md5sum` and `zstd` . `zstd` is needed because the dense inputs in the data package are zstandard-compressed and are inflated on demand (e.g. `sudo apt install zstd` on Debian/Ubuntu). `runme.sh` checks for these tools before starting.

1. Get the Code (with the `mm-repair` submodule)

The CPU-side grammar compressor lives in the **`mm-repair` submodule**, so it must be initialized — a fresh `git clone` leaves `mm-repair/` empty otherwise:

```
git clone https://github.com/ftosoni/g-mm-repair.git
cd g-mm-repair
git submodule update --init --recursive
```

(If you cloned without `--recurse-submodules` , just run the last line inside the repo.)

2. Compile the GPU Engine and Baselines

To compile the CUDA-accelerated sweep engine and the cuSPARSE benchmark:

```
cd gpu-engine
make clean && make
make cusparse_test
cd ..
```

> **GPU architecture.** `gpu-engine/Makefile` targets `sm_121` by default (the Grace-Blackwell GB10 used in the paper). On any other GPU pass your compute capability, e.g. `make CUDA_ARCH=sm_90` (Hopper), `sm_89` (Ada), `sm_80` (Ampere), or `make CUDA_ARCH=native` for the GPU of the build machine; otherwise the binaries will not run.
>
> **CUDA toolkit.** `nvcc` is taken from `$CUDA_HOME/bin` if `CUDA_HOME` is set, else from `PATH`, else from `/usr/local/cuda/bin`; `make NVCC=/path/to/nvcc` overrides it. The scripts (`runme.sh`, `reproduce.sh`, ...) honour `CUDA_HOME` as well.

3. Compile the `mm-repair` Toolchain

Build the CPU grammar compressor **and the helpers it drives** — the matrix→CSRV converter (`csvmat2csrv`), RePair (`brepair/irepair0`), and the integer/ANS encoders. `make all` produces the `matrepair` constructor plus everything it invokes:

```
cd mm-repair
make all
cd ..
```

> **Dependency.** `mm-repair` links against SDSL-lite (used for the packed `.iv` integer vectors). Install it first — see `mm-repair/Readme.md` for the exact prerequisites and build details. (For the manuscript's build-time column only, `re32mm` can be rebuilt with detailed timing: `make re32mm CFLAGS="-Wall -std=c99 -g -O3 -DDETAILED_TIMING"`.)

> **Memory.** By default `matrepair` lets RePair use 90% of the RAM, capped by the cgroup memory limit when running under a batch scheduler (e.g. Slurm) or in a container; `matrepair -m <MB>` sets the budget explicitly. The grammars used in the paper are already in the Zenodo package, so rebuilding them is only needed for the build-cost table (`reproduce.sh grammar`).

4. Run a Quick Test

For validation and benchmarking of a specific compressed matrix, run the test driver by passing the matrix base path and dimensions:

```
./gpu-engine/gpu_test <matrix_base_path> <rows> <cols> [iterations]
```

🔢 Worked Example: multiply a small matrix by a chosen vector

A full, self-contained walkthrough that goes from a plain-text matrix to a verified $y = Mx$ — useful to see the whole pipeline (construction → grammar → engine) end-to-end. It assumes the three build steps above are done.

Take the 8×7 matrix M and the column vector $x = [2, 1, 1, 2, 1, 1, 1]^\mathsf{T}$. By hand, $y = Mx = [12.9,\ 20.7,\ 20.7,\ 12.9,\ 20.7,\ 19.3,\ 19.3,\ 12.9]^\mathsf{T}$.

```
mkdir -p examples

# 1) the matrix as CSV: comma-separated doubles, one row per line.
#    Zeros are allowed — they are dropped during construction (they contribute nothing to y).
cat > examples/M.csv <<'EOF'
2.3,0,1.2,1.2,1.2,2.3,1.2
1.2,4.5,3.4,1.2,1.2,2.3,4.5
1.2,4.5,3.4,1.2,1.2,2.3,4.5
2.3,0,1.2,1.2,1.2,2.3,1.2
1.2,4.5,3.4,1.2,1.2,2.3,4.5
0,4.5,3.4,2.3,2.3,0,4.5
0,4.5,3.4,2.3,2.3,0,4.5
2.3,0,1.2,1.2,1.2,2.3,1.2
EOF

# 2) build the grammar-compressed representation (C, R, V):
#    -> examples/M.csv.val  examples/M.csv.vc.C  examples/M.csv.vc.R
./mm-repair/matrepair -r examples/M.csv 8 7

# 3) the input vector x = [2,1,1,2,1,1,1] as little-endian float32.
#    This is the exact binary format shared by the engine's XVEC option and every baseline.
python3 -c "import numpy; numpy.asarray([2,1,1,2,1,1,1],dtype='<f4').tofile('examples/x.bin')"

# 4) run the engine on the chosen x. XVEC injects it; CROSSCHECK dumps the used x and the
#    resulting y next to it (examples/out.plustimes.{x,y}).
XVEC=examples/x.bin CROSSCHECK=examples/out ./gpu-engine/gpu_test examples/M.csv 8 7 1 repair

# 5) print y
python3 -c "import numpy; print(numpy.fromfile('examples/out.plustimes.y'))"
```

You should see `XVEC: loaded fixed x (7 float32) ...`, then `SUCCESS: GPU and CPU reference results match`, and finally:

```
[12.9 20.7 20.7 12.9 20.7 19.3 19.3 12.9]
```

matching the hand computation (tiny `~1e-6` differences are expected — the engine accumulates in `float32`).

Optional — cross-check against cuSPARSE on the same `x`. The engine already dumped `examples/out.plustimes.{x,y}`; an independent implementation reloads that exact vector and must agree:

```
CROSSCHECK=examples/out ./gpu-engine/cusparse_test examples/M.csv 8 7 1
# -> CROSSCHECK cusparse vs engine: ... SUCCESS
```

Notes:

- Without `XVEC` the engine draws `x` at random (still verified bit-for-bit against its CPU oracle) — `XVEC` only pins a *specific* vector.
- Prepend `SEMIRING=boolean` or `SEMIRING=tropical` to exercise the other semirings (pick `x` in the matching domain; the dump becomes `out.boolean.y` / `out.tropical.y`).
- The `examples/` folder is a throwaway scratch dir; delete it when done.

📦 Data Availability

All datasets (1000 Genomes genotypes, synthetic haplotypes, `crossover_synth`, the Wikidata relations, and the billion-edge Software Heritage graph) and the RePair grammars the engine consumes are archived on Zenodo:

DOI 10.5281/zenodo.22677746

> **DOI:** `10.5281/zenodo.22677746` — concept DOI, always resolves to the latest version (camera-ready version: `10.5281/zenodo.22677747`).

Download the record, extract the three tar archives, and verify integrity:

```
mkdir -p zenodo && cd zenodo
REC=https://zenodo.org/api/records/22677747/files
for f in genotypes.tar wikidata.tar swh.tar README.txt MANIFEST.md5; do
  curl -L -o "$f" "$REC/$f/content"
done
for t in genotypes.tar wikidata.tar swh.tar; do tar xf "$t"; done   # -> genotypes/ wikidata/ swh/
md5sum -c MANIFEST.md5                                              # expect: all files OK
cd ..
```

This lands everything in `./zenodo/`, exactly where `reproduce.sh` looks (override with `ZENODO_DIR=/path/to/package`). File formats and how to rebuild every dataset from scratch are documented in REPRODUCIBILITY.md.

📊 Reproducibility

`reproduce.sh` is the single source of truth. It runs each experiment into `manuscript/logs/`, then regenerates every paper table and figure from those logs via `extract_results.py`. With the Zenodo package in `./zenodo/`, on a CUDA host:

```
./reproduce.sh all      # struct + time + space + spmm + graph, then extract + plot
```

For a single-command run from a fresh clone — initialise the submodule, build everything, fetch + verify the Zenodo data, then run the pipeline above — use the top-level wrapper:

```
./runme.sh              # submodule init -> build -> download+verify data -> reproduce.sh all
```

Besides building, `runme.sh` creates `./gbvenv`, a Python virtualenv with `python-graphblas` for the SuiteSparse:GraphBLAS baseline of Table 5.2 (needs `python3-venv` and network access). When calling `reproduce.sh` directly, create it first: `python3 -m venv gbvenv && ./gbvenv/bin/pip install python-graphblas`.

Each stage produces a specific paper artifact:

`reproduce.sh <stage>`	In `all`	Paper output
`struct`	✔	Structural sizes (nonterminals $\lvert\mathcal{R}\rvert$, depth L, width w^*) — **Table 4.1** (genotypes) & **Table B.2** (Wikidata)
`time`	✔	Genotype average time/vector: engine vs CPU sweeps, `mm-repair`, cuSPARSE — **Table 4.2**
`space`	✔	Genotype space & energy vs cuSPARSE, incl. `crossover_synth` — **Table 4.3** & **Figure 4.3**
`spmm`	✔	Batched right product $Y = MX$: engine vs cuSPARSE — **Table A.1** & **Figure A.1**
`graph`	✔	Wikidata Boolean & Tropical product vs GraphBLAS — **Table 5.2**
`extract`	✔	Re-derive every table + figure datum from the existing logs (no recompute)
`plot`	✔	Compile the two data-driven figures (4.3, A.1)
`graphscale`	—	Large-scale graphs, 10M–1.2G edges, incl. SWH — **Table 5.1** (heavy, on demand)
`grammar`	—	Offline RePair grammar-build cost — **Table B.1** (heavy, on demand)
`crosscheck`	—	Cross-implementation correctness (engine vs CPU / cuSPARSE / GraphBLAS / `mm-repair`); backs the bit-for-bit claims (heavy, on demand)

For per-table manual commands, baseline setup (SuiteSparse:GraphBLAS, cuGraph), and full dataset provenance, see REPRODUCIBILITY.md — an optional deep-dive; you don't need it to reproduce the results.

📖 Citation

If you use this software or its datasets, please cite the paper:

> Francesco Tosoni and Gabriele Mencagli. "Streaming Right Multiplication over Grammar-Compressed Matrices: A Memory-Bounded GPU Engine for Genotype and Graph Data." In *Proceedings of the SIAM Symposium on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2027. To appear.

```bibtex
@inproceedings{tosoni2027streaming,
  title     = {Streaming Right Multiplication over Grammar-Compressed Matrices: A Memory-Bounded {GPU} Engine f
  author    = {Tosoni, Francesco and Mencagli, Gabriele},
  booktitle = {Proceedings of the SIAM Symposium on Algorithm Engineering and Experiments (ALENEX)},
  year      = {2027},
  publisher = {SIAM},
  note      = {To appear},
}
```

The dataset package is archived on Zenodo, DOI 10.5281/zenodo.22677746. The version of the code evaluated for ALENEX 2027 is archived on Zenodo, DOI 10.5281/zenodo.23139248. The source code is also archived at Software Heritage; cite this exact snapshot by its SWHID:

```
swh:1:dir:ef2c5f738ef173b38a2c7f1edc8969d06159785d;origin=https://github.com/ftosoni/g-mm-repair;visit=swh:1:snp:f6872312e0713082be0ecd7209b57db4008f8df6;anchor=swh:1:rev:f007357c261ff4f8b1314ccd0c486eca88cfd8b3
```

See `CITATION.cff` for machine-readable citation metadata.

License

This project is licensed under the Apache License, Version 2.0. See the LICENSE file for details.